

c program for aligned malloc with answers to alignment

1. fragmentation
2. free-list management
3. splitting
4. coalescing
5. thread safety

Absolutely. This is a **very good C/Linux systems interview problem**, especially for NVIDIA/Intel/embedded roles. The interviewer may ask you to implement `aligned_malloc()` and then extend it into a small allocator covering **fragmentation, free lists, splitting, coalescing, and thread safety**.

1. Basic `aligned_malloc()` implementation

First, understand the simpler problem:

Allocate `size` bytes such that the returned address is aligned to `alignment`.

For example:

```
alignment = 16
```

Valid addresses:

```
0x1000
```

```
0x1010
```

```
0x1020
```

```
0x1030
```

```
...
```

Invalid:

```
0x1001
```

```
0x1008
```

```
0x100F
```



Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

void *aligned_malloc(size_t size, size_t alignment)
{
    if (size == 0)
        return NULL;

    /* Alignment must be power of 2 */
    if (alignment == 0 || (alignment & (alignment - 1)) != 0)
        return NULL;

    /*
     * Need:
     *
     *   extra space for alignment
     *   extra space to store original malloc pointer
     */
    size_t total = size + alignment - 1 + sizeof(void *);

    if (total < size)          /* overflow check */
        return NULL;

    void *raw = malloc(total);

    if (raw == NULL)
        return NULL;

    /*
     * Leave space at beginning for storing raw pointer.
     */
    uintptr_t start = (uintptr_t)raw + sizeof(void *);

    /*
     * Round start UP to next alignment boundary.
     */
    uintptr_t aligned =
        (start + alignment - 1) & ~(uintptr_t)(alignment - 1);

    /*
     * Store original malloc pointer immediately
     * before aligned pointer.
     */
    ((void **)aligned)[-1] = raw;
```

</> C



And corresponding free :

```
void aligned_free(void *ptr)
{
    if (ptr == NULL)
        return;

    void *raw = ((void **)ptr)[-1];
```

</> C



Test

```
int main(void)
{
    size_t alignment = 64;

    void *p = aligned_malloc(100, alignment);

    if (p == NULL)
        return 1;

    printf("ptr      = %p\n", p);
    printf("address  = 0x%lx\n", (unsigned long)(uintptr_t)p);
    printf("remainder = %lu\n",
        (unsigned long)((uintptr_t)p % alignment));

    aligned_free(p);
```

</> C



Expected:

remainder = 0



So:

(uintptr_t)p % 64 == 0



2. How the alignment calculation works

This line is extremely important for an interview:

```
</> C
```



Suppose:

```
start      = 0x1007  
alignment = 16
```



We want:

```
0x1010
```



because:

```
0x1007 → 0x1010
```



Calculation:

```
start + alignment - 1
```



```
0x1007 + 0x0F  
= 0x1016
```

Then:

```
~(alignment - 1)
```



```
alignment = 16 = 0x10
```

```
alignment - 1 = 0x0F
```

```
~0x0F = ...F0
```

Therefore:

```
0x1016
&
...F0
=
0x1010
```



Interview question

Q: Why does this work only when alignment is a power of two?

Because:

```
alignment = 16
binary     = 10000
alignment-1= 01111
```



The mask:

```
~(alignment - 1)
```



clears exactly the lower bits needed for alignment.

For example:

```
8  -> 00001000
16 -> 00010000
32 -> 00100000
64 -> 01000000
```



3. But this is NOT a full memory allocator

The above implementation uses:

```
</> C
```



underneath.

So it doesn't demonstrate:

1. fragmentation

2. free-list management
3. splitting
4. coalescing
5. thread safety

If the interviewer says:

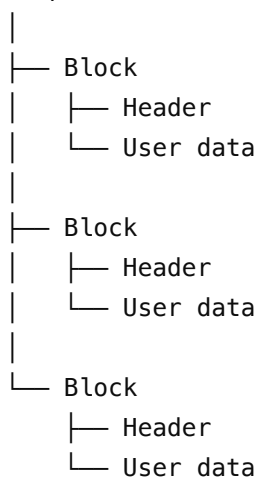
"Implement a simple malloc/free allocator and explain these concepts."

then you need to go one level deeper.

4. Memory allocator architecture

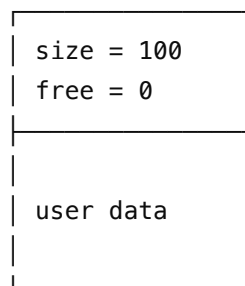
A simple allocator can manage a memory region like:

Heap



For example:

allocated



The header might be:

```
struct block {
    size_t size;
    int free;

    struct block *next;
}
```

</> C



5. Free-list management

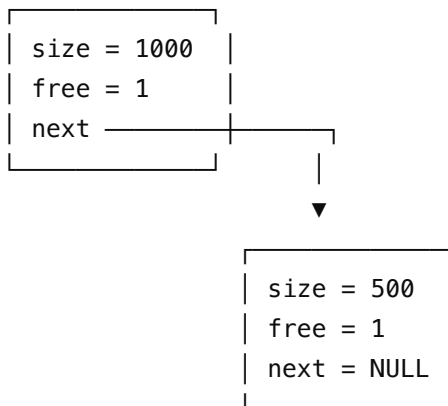
Instead of throwing freed memory away, maintain a linked list of free blocks.

Example:

Free list



head



When the application requests:

</> C



allocator searches:

1000-byte free block



and uses it.

But what happens to the remaining 600 bytes?

That leads to **splitting**.

6. Splitting

Suppose we have:

Free block = 1000 bytes
Application requests = 400 bytes



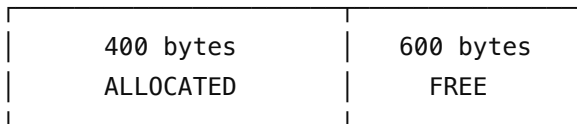
Instead of consuming all 1000 bytes:

Before:

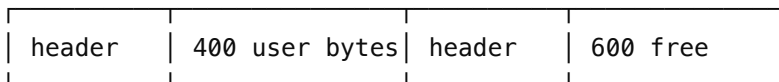


split it:

After:



In reality, headers consume some space:



Why don't we always split?

Because creating a tiny free block is useless.

For example:

```
free block = 100 bytes
request    = 96 bytes
```



If the header requires 32 bytes:

$$100 - 96 - 32 < 0$$


We can't create a useful second block.

So typically:

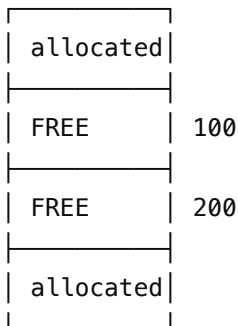
```
</> C
```



7. Coalescing

Now the opposite problem.

Suppose memory looks like:



There are:

$$100 + 200 = 300 \text{ bytes}$$


of free memory.

But they're separate.

If somebody requests:

</> C

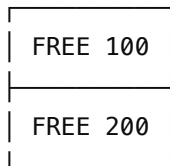


you might fail to find one contiguous 250-byte block.

Coalescing

Merge adjacent free blocks:

Before:



After:



This is **coalescing**.

8. External fragmentation

This is a very common interview question.

Suppose:

Total free memory = 1000 bytes



but it looks like:

FREE 100
USED
FREE 200
USED
FREE 300
USED
FREE 400



Total:

```
100 + 200 + 300 + 400 = 1000
```



But:

```
</> C
```



can fail because there is no contiguous 800-byte region.

That's:

External fragmentation

Memory exists, but it's fragmented into separate free blocks.

9. Internal fragmentation

Different problem.

Suppose:

```
</> C
```



but allocator gives you:

```
32 bytes
```



because of alignment.

Then:

```
requested = 25  
allocated = 32
```



```
wasted = 7
```

That's:

Internal fragmentation

The wasted space is **inside the allocated block**.

Typical causes:

```
alignment
minimum allocation size
metadata/header
size-class rounding
```



Easy interview distinction

Remember:

```
Internal fragmentation
  ↓
waste INSIDE allocated block

External fragmentation
  ↓
free space scattered OUTSIDE allocated blocks
```



10. Simple allocator structure

A basic allocator can therefore look like:

```
struct block {
    size_t size;
    int free;

    struct block *next;
}
```

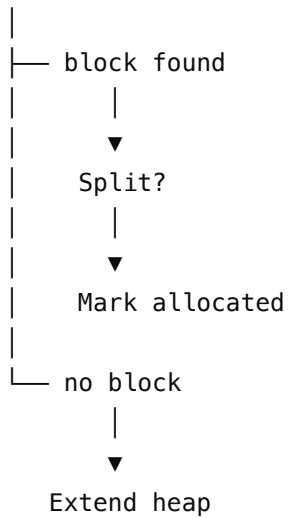
</> C



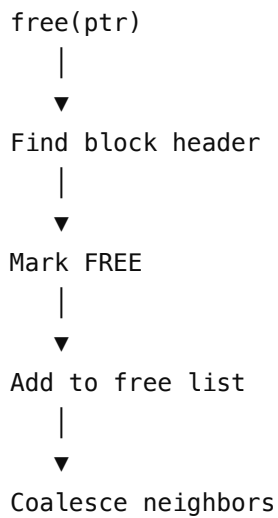
Allocation:

```
malloc(size)
  |
  ▼
Search free list
```



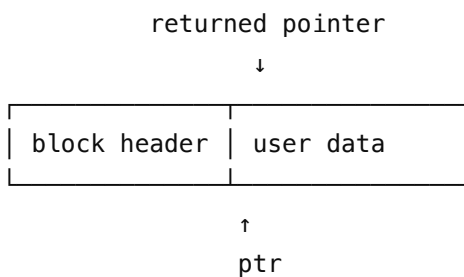


Free:



11. Example free() concept

If user receives:



The header is immediately before ptr .

Therefore:

```
</> C
```



Then:

```
</> C
```



Now you can put it back into the free list.

This is closely related to the Linux kernel's `container_of()` concept you have been studying:

```
</> C
```

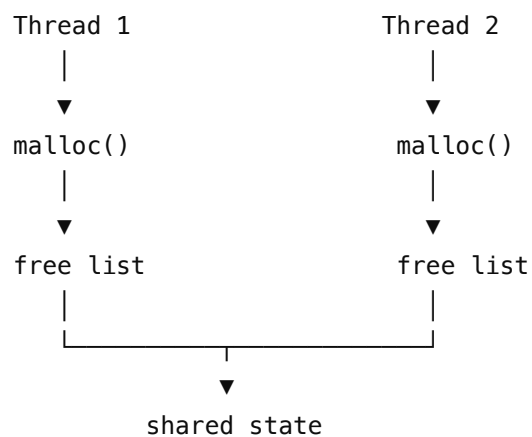


Both rely on knowing the relationship between an embedded object/member and its containing structure.

12. Thread safety

This is the fifth topic the interviewer may ask.

Consider:



If both threads modify:

</> C



simultaneously, you can corrupt the linked list.

Example:

Thread 1:
free_list->next = A



Thread 2:
free_list->next = B

One update can overwrite the other.

13. Mutex solution

Protect allocator metadata:

</> C



Then:

```
void *my_malloc(size_t size)
{
    pthread_mutex_lock(&allocator_lock);

    /* search free list */
    /* split block */
    /* update metadata */

    pthread_mutex_unlock(&allocator_lock);
}
```

</> C



And:

```
void my_free(void *ptr)
{
    pthread_mutex_lock(&allocator_lock);

    /* modify free list */
    /* coalesce */
}
```

</> C



Important interview point

Don't hold the lock longer than necessary.

You want:

```
lock
↓
modify allocator metadata
↓
unlock
```



rather than doing unrelated expensive work while holding the lock.

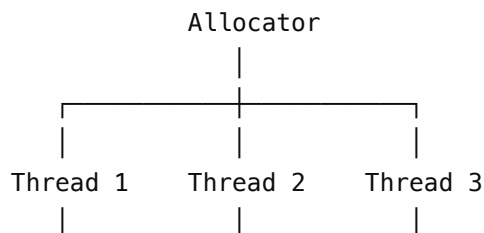
14. Better allocator: per-thread arenas

If the interviewer asks:

"How would you improve scalability?"

A single global mutex is simple but can become a bottleneck.

Instead:



Arena 1	Arena 2	Arena 3
free list	free list	free list

Each thread can primarily use its own arena.

This reduces lock contention.

Modern allocators use much more sophisticated approaches, including:

- multiple arenas
- size classes
- thread-local caches
- segregated free lists
- page-based allocation

15. Interview cheat sheet

If NVIDIA asks:

Q: What is fragmentation?

Answer:

Fragmentation is wasted memory caused by the way memory is allocated and freed. Internal fragmentation is unused space inside allocated blocks, while external fragmentation is free memory divided into separate non-contiguous blocks.

Q: What is a free list?

Answer:

A free list is a data structure, commonly a linked list, that tracks memory blocks currently available for allocation.

Q: What is splitting?

Answer:

Splitting takes a larger free block and divides it into an allocated block and a

remaining free block, reducing wasted space.

Q: What is coalescing?

Answer:

Coalescing merges adjacent free blocks into a larger free block, reducing external fragmentation.

Q: Why is coalescing important?

```
FREE 100  
FREE 200  
FREE 300
```



can become:

```
FREE 600
```



so a large allocation can succeed.

Q: How do you make malloc thread-safe?

Answer:

Protect shared allocator metadata such as free lists with a mutex or another synchronization mechanism. For scalability, more advanced allocators can use per-thread caches or multiple arenas to reduce lock contention.

Q: Why can't you simply call `free(ptr)` for an aligned pointer?

Because:

```
malloc()  
↓  
raw = 0x1003
```



```
aligned = 0x1010
```

The pointer returned to the caller is:

```
0x1010
```



but `free()` expects:

```
0x1003
```



the **original pointer returned by** `malloc()` .

That's why we store:

```
</> C
```



and later:

```
</> C
```



The key mental picture

For your interview, remember this:

```
aligned_malloc()
```



```
malloc(raw)
```



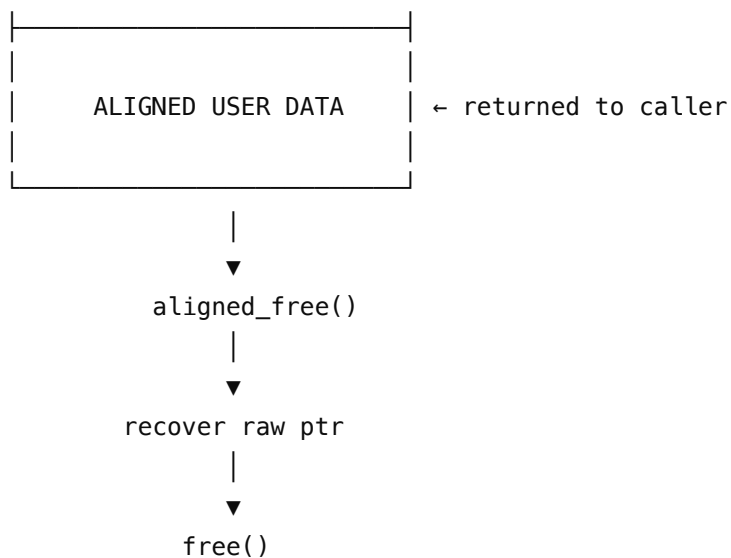
```
extra alignment space  
+ metadata
```

```
original malloc pointer
```

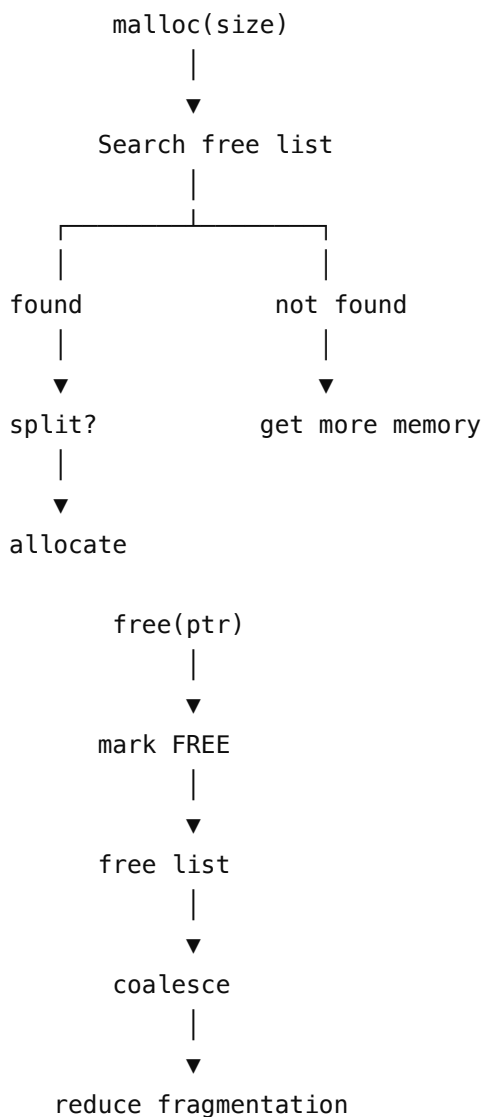
```
alignment padding
```

Upgrade





And for a **full allocator**:



For an NVIDIA-style C/system interview, I would be prepared to **write the aligned allocator from scratch**, then explain **why the `uintptr_t` alignment calculation works**, **how `free()` recovers the original pointer**, and **how you would extend it into a thread-safe free-list allocator with splitting/coalescing**.



ChatGPT can make mistakes. Check important info.